

AMPL Quick Reference Guide

Graduate Optimization Course | Northern Illinois University

Table of Contents

1.	Core Syntax	Sets, Parameters, Variables, Objectives, Constraints
2.	Data File Patterns	Sets, Scalars, 1D/2D Parameters, Tables
3.	Math to AMPL Translation	Summations, Constraints, Common Patterns
4.	Common Modeling Patterns	Resource, Blending, Transport, Assignment
5.	Python Integration	Setup, Workflow, Results, Data Injection
6.	Debugging Guide	Syntax Errors, Model Errors, Debugging Commands
7.	Quick Reference Tables	Operators, Functions, Set Operations

1. Core Syntax

1.1 Sets

Sets define the indices used throughout your model.

```
# Basic set (members defined in .dat file)
set PRODUCTS;

# Ordered set (enables prev, next, first, last)
set MONTHS ordered;

# Set of pairs (arcs in a network)
set ARCS within {NODES, NODES};

# Derived set with condition
set ACTIVE := {j in PRODUCTS: demand[j] > 0};

# Cross product
set ROUTES := ORIGINS cross DESTINATIONS;
```

Key Set Functions:

Function	Description	Example
card(S)	Number of elements	param n := card(PRODUCTS);
ord(i)	Position (1-based)	ord(t) > 1
first(S)	First element	first(MONTHS)
last(S)	Last element	last(MONTHS)
prev(i)	Previous element	inventory[prev(t)]
next(i)	Next element	demand[next(t)]
member(k,S)	k-th element	member(3, MONTHS)

1.2 Parameters

Parameters are known constants (input data).

```
# Scalar parameter
param budget;

# Indexed parameter
param demand {PRODUCTS};

# Two-dimensional parameter
param cost {ORIGINS, DESTINATIONS};

# Parameter with default value
param min_order {PRODUCTS} default 0;

# Computed parameter
param total_demand := sum {j in PRODUCTS} demand[j];

# Parameter with bounds check
param capacity {MACHINES} > 0;
```

1.3 Decision Variables

Variables are what the solver determines.

```

# Non-negative continuous variable
var Produce {PRODUCTS} >= 0;

# Variable with upper bound
var Ship {ORIGINS, DESTINATIONS} >= 0, <= capacity;

# Variable with parameter bounds
var Make {j in PRODUCTS} >= min_prod[j], <= max_prod[j];

# Integer variable
var Assign {WORKERS, TASKS} >= 0 integer;

# Binary variable
var Select {PROJECTS} binary;

# Free variable (can be negative)
var Deviation {GOALS};

```

1.4 Objective Function

Every model has exactly one objective.

```

# Maximization
maximize Total_Profit:
    sum {j in PRODUCTS} profit[j] * Produce[j];

# Minimization
minimize Total_Cost:
    sum {i in ORIGINS, j in DESTINATIONS} cost[i,j] * Ship[i,j];

```

1.5 Constraints

Use "subject to" (or "s.t.") to define constraints.

```

# Single constraint
subject to Budget_Limit:
    sum {j in PRODUCTS} cost[j] * Produce[j] <= budget;

# Family of constraints (one per index)
subject to Capacity {i in MACHINES}:
    sum {j in PRODUCTS} time[i,j] * Produce[j] <= available[i];

# Equality constraint
subject to Balance:
    sum {i in ORIGINS} Supply[i] = sum {j in DESTINATIONS} Demand[j];

# Constraint with conditional indexing
subject to Min_Production {j in PRODUCTS: required[j] = 1}:
    Produce[j] >= min_qty[j];

```

2. Data File Patterns

2.1 Set Definitions

```
# Simple set
set PRODUCTS := Chairs Tables Desks;

# Set on multiple lines
set MONTHS := Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec;

# Set of pairs
set ARCS := (A,B) (A,C) (B,C) (B,D) (C,D);
```

2.2 Scalar Parameters

```
param budget := 50000;
param tax_rate := 0.08;
param planning_horizon := 12;
```

2.3 One-Dimensional Parameters

```
# List format
param demand :=
    Chairs    100
    Tables    50
    Desks     75;

# Single line format
param supply := Seattle 400 Denver 300 Chicago 500;
```

2.4 Two-Dimensional Parameters

Table Format (preferred for dense data):

```
param cost:           Chicago Denver Seattle :=
    Portland          2.5      1.8      1.2
    Sacramento        2.8      2.0      1.5
    LosAngeles        2.2      1.9      2.1;
```

List Format (preferred for sparse data):

```
param distance :=
    [Portland, Chicago]    2000
    [Portland, Denver]    1200
    [Sacramento, Chicago] 1850;
```

2.5 Multiple Parameters in One Table

```
param:           demand    price    min_order :=
    Chairs       100        45        10
    Tables       50         120        5
    Desks        75         200        5;
```

3. Mathematical Notation to AMPL Translation

3.1 Summations

Mathematical Form	AMPL Code
Sum of $c_j * x_j$ over j in J	sum {j in J} c[j] * x[j]
Sum from $i=1$ to n	sum {i in 1..n} a[i]
Conditional sum ($d_j > 0$)	sum {j in J: d[j] > 0} x[j]
Double sum over i and j	sum {i in I, j in J} c[i,j] * x[i,j]
Sum over arc pairs	sum {(i,j) in ARCS} x[i,j]

3.2 Constraints

Mathematical Form	AMPL Code
Sum $a_{ij} * x_j \leq b_i$ for all i	subject to C {i in I}: sum {j in J} a[i,j] * x[j] <= b[i];
$x_j \geq 0$ for all j	var x {J} >= 0;
Sum of $x_j = 1$	subject to Total: sum {j in J} x[j] = 1;
$l_j \leq x_j \leq u_j$	var x {j in J} >= l[j], <= u[j];
x_j is binary	var x {J} binary;
x_j is non-neg integer	var x {J} >= 0 integer;

3.3 Common Formulation Patterns

Resource Constraint:

```
subject to Resource {i in RESOURCES}:  
    sum {j in PRODUCTS} usage[i,j] * Produce[j] <= available[i];
```

Flow Balance:

```
subject to Balance {i in NODES}:  
    sum {(i,j) in ARCS} Flow[i,j] - sum {(j,i) in ARCS} Flow[j,i] = supply[i];
```

Inventory Balance:

```
subject to Inv_Balance {t in PERIODS: ord(t) > 1}:  
    Inventory[t] = Inventory[prev(t)] + Production[t] - demand[t];
```

Big-M Linking Constraint:

```
subject to Linking {j in FACILITIES}:  
    Production[j] <= big_M * Open[j];
```

4. Common Modeling Patterns

4.1 Resource Allocation

```
set PRODUCTS;
set RESOURCES;
param profit {PRODUCTS};
param usage {RESOURCES, PRODUCTS};
param available {RESOURCES};

var Produce {PRODUCTS} >= 0;

maximize Total_Profit:
    sum {j in PRODUCTS} profit[j] * Produce[j];

subject to Capacity {i in RESOURCES}:
    sum {j in PRODUCTS} usage[i,j] * Produce[j] <= available[i];
```

4.2 Transportation

```
set ORIGINS;
set DESTINATIONS;
param supply {ORIGINS};
param demand {DESTINATIONS};
param cost {ORIGINS, DESTINATIONS};

var Ship {ORIGINS, DESTINATIONS} >= 0;

minimize Total_Cost:
    sum {i in ORIGINS, j in DESTINATIONS} cost[i,j] * Ship[i,j];

subject to Supply {i in ORIGINS}:
    sum {j in DESTINATIONS} Ship[i,j] <= supply[i];

subject to Demand {j in DESTINATIONS}:
    sum {i in ORIGINS} Ship[i,j] >= demand[j];
```

4.3 Assignment

```
set WORKERS;
set TASKS;
param cost {WORKERS, TASKS};

var Assign {WORKERS, TASKS} binary;

minimize Total_Cost:
    sum {w in WORKERS, t in TASKS} cost[w,t] * Assign[w,t];

subject to One_Task {w in WORKERS}:
    sum {t in TASKS} Assign[w,t] = 1;

subject to One_Worker {t in TASKS}:
    sum {w in WORKERS} Assign[w,t] = 1;
```

4.4 Multi-Period Production Planning

```
set PERIODS ordered;
param demand {PERIODS};
param prod_cost {PERIODS};
param hold_cost {PERIODS};
param max_production {PERIODS};
param init_inventory;

var Produce {t in PERIODS} >= 0, <= max_production[t];
var Inventory {PERIODS} >= 0;

minimize Total_Cost:
    sum {t in PERIODS} (prod_cost[t]*Produce[t] + hold_cost[t]*Inventory[t]);

subject to Init_Balance:
    Inventory[first(PERIODS)] = init_inventory
        + Produce[first(PERIODS)] - demand[first(PERIODS)];

subject to Balance {t in PERIODS: ord(t) > 1}:
    Inventory[t] = Inventory[prev(t)] + Produce[t] - demand[t];
```

5. Python Integration with amplpy

5.1 Setup in Google Colab

```
# Install
!pip install -q amplpy

from amplpy import AMPL, ampl_notebook

# Initialize with license and solvers
ampl_notebook(
    modules=["highs"], # Add "gurobi", "cplex" if available
    license_uuid="your-license-uuid"
)
```

5.2 Basic Workflow

```
# Create model instance
model = AMPL()

# Load model and data files
model.read('mymodel.mod')
model.read_data('mydata.dat')

# Set solver and solve
model.option['solver'] = 'highs'
model.solve()

# Check status
status = model.get_value('solve_result')
print(f"Status: {status}")
```

5.3 Accessing Results

```
# Objective value
obj_value = model.obj['Total_Cost'].value()

# Variable values
for j in model.set['PRODUCTS'].members():
    print(f"{j}: {model.var['Produce'][j].value()}")

# Get as pandas DataFrame
df = model.var['Ship'].get_values().to_pandas()

# Shadow prices (dual values)
shadow = model.con['Capacity']['Labor'].dual()

# Constraint body (LHS value)
```

```
used = model.con['Capacity']['Labor'].body()
```


5.4 Injecting Data from Python

```
# Set members
model.set['PRODUCTS'] = ['A', 'B', 'C']

# Scalar parameter
model.param['budget'] = 50000

# Indexed parameter (dict)
model.param['demand'] = {'A': 100, 'B': 150, 'C': 200}

# Two-dimensional parameter
costs = {
    ('Plant1', 'Customer1'): 10,
    ('Plant1', 'Customer2'): 15,
    ('Plant2', 'Customer1'): 12,
    ('Plant2', 'Customer2'): 8,
}
model.param['cost'] = costs
```

5.5 Sensitivity Analysis Loop

```
results = []
for budget in [40000, 50000, 60000, 70000]:
    scenario = AMPL()
    scenario.read('model.mod')
    scenario.read_data('data.dat')
    scenario.param['budget'] = budget
    scenario.option['solver'] = 'highs'
    scenario.solve()

    results.append({
        'budget': budget,
        'profit': scenario.obj['Profit'].value(),
        'status': scenario.get_value('solve_result')
    })

df_results = pd.DataFrame(results)
```

6. Debugging Guide

6.1 Common Syntax Errors

Error Message	Likely Cause	Fix
syntax error	Missing semicolon	Add ; at end
X is not defined	Using before declaring	Move declaration up
X is already defined	Duplicate declaration	Remove duplicate
invalid subscript	Wrong indices	Check set membership
no value for X	Missing data	Add to .dat file

6.2 Model Errors

Infeasible Problem:

- Check for conflicting constraints (e.g., demand > supply)
- Verify data values are correct
- Add slack variables to identify problematic constraints

Unbounded Problem:

- Missing constraints (e.g., no capacity limit)
- Variable bounds missing
- Objective coefficient sign error

6.3 Debugging Commands

```
# Display model components
display PRODUCTS;
display demand;
display Produce;

# Show expanded constraints
expand Capacity;
expand Capacity['Labor'];

# Check constraint values
display {i in RESOURCES} Capacity[i].body, Capacity[i].slack;
```

7. Quick Reference Tables

7.1 Arithmetic Operators

Operator	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
^	Exponentiation	$a ^ 2$
mod	Modulo	$a \text{ mod } b$
div	Integer division	$a \text{ div } b$

7.2 Built-in Functions

Function	Description
abs(x)	Absolute value
max(a,b)	Maximum of two values
min(a,b)	Minimum of two values
round(x)	Round to nearest integer
floor(x)	Round down
ceil(x)	Round up
sqrt(x)	Square root
exp(x)	Exponential
log(x)	Natural logarithm

7.3 Set Operations

Operation	Syntax	Description
Union	$A \cup B$	All elements in A or B
Intersection	$A \cap B$	Elements in both A and B
Difference	$A - B$	Elements in A but not B
Cross product	$A \times B$	All pairs (a,b)
Membership	$i \in A$	True if i is in A
Subset	$A \subseteq B$	A is subset of B

7.4 Solver Status Codes

Status	Meaning	Action
solved	Optimal solution found	Extract results
infeasible	No feasible solution	Check constraints
unbounded	Objective unbounded	Add constraints/bounds
limit	Hit iteration/time limit	Increase limits
failure	Solver error	Check model/data

Additional Resources

- **AMPL Book (Free PDF):** <https://ampl.com/wp-content/uploads/BOOK.pdf>
- **AMPL Documentation:** <https://dev.ampl.com>
- **amplpy Python API:** <https://amplpy.ampl.com>
- **Free Academic License:** <https://ampl.com/ce> or <https://ampl.com/courses>
- **Support Forum:** <https://discuss.ampl.com>

Last updated: February 2026